



How to **develop for Hue?**

Develop

[Getting started](#)[Application Design Guidance](#)[Hue API V1](#)[Hue API V2](#)[Hue Entertainment](#)[Tools and SDKs](#)[Overview](#)[Hue Entertainment Blog](#)[Hue Entertainment API](#)[Hue EDK Effect Creation](#)[Light Effect Guide](#)[Hue HDMI Sync Box API](#)

Hue HDMI Sync Box API

Introduction

The Philips Hue HDMI Sync Box lets you sync your Hue lights with your HDMI based TV content, by placing it in between your TV and a maximum of 4 HDMI sources. It can be controlled through a HTTP JSON API much like the Hue Bridge. This page explains how to use the Sync Box API. Note that before apiLevel 7 this API was in beta and breaking changes still occurred, so if any lower apiLevel is detected the user should be notified to upgrade first.

Getting Started

There are 4 mandatory steps before using the API.

1. Set up Sync Box with the official Hue Sync iOS/Android app
2. Discover Sync Box through mDNS
3. Create API registration to obtain access token
4. Use API on HTTPS with access token

In this getting started we will skip Step 2 by getting the Hue Sync Box IP address from the Hue Sync mobile app, and use insecure communication by not validating the device certificate. This is only for quickly trying out the API, a production app should follow all steps and only use secure communication.

On this page:

[Introduction](#)[Getting Started](#)[Device Discovery](#)[Registration](#)[HTTPS](#)[Example Configuration](#)[General Concepts](#)[Resource Table \(apiLevel 10\)](#)[Responses](#)

Open the Hue app, navigate to the Sync Box on the Sync tab and press ...
> Device > Network info to find its IP address.

From the same wifi network as the Sync Box is on, send the following request to confirm response from the Sync Box (the -k option disables certificate validation for now):

```
curl -k -X GET https://<ip>/api/v1/device
```

Now execute the following request to attempt to create an API registration:

```
curl -k -d '{"appName":"curl", "instanceName":"<yourname>"}' -X POST https://<ip>/api/v1/registrations
```

And observe the response indicating that the device button is not yet pressed:

```
{"code":16,"message":"Invalid State"}
```

Within 5 seconds of the response, hold the device button until the led blinks green (~3 seconds) and release.

Within 5 seconds of releasing, send the same request again and observe a response with an accessToken like:

```
{"registrationId":"1","accessToken":"<token>"}
```

Now copy the accessToken and use it in the following request to retrieve the Sync Box device state.

```
curl -k -H "Authorization: Bearer <token>" -X GET https://<ip>/api/v1
```

Finally, make sure some content is playing from the current HDMI source, and use the following request to enable video sync mode:

```
curl -k -d '{"mode":"video"}' -H "Authorization: Bearer <token>" -X PUT https://<ip>/api/v1/execution
```

Now that we've seen a basic example of using the API, the next sections will explain how to properly implement the steps in an application, as well as provide a full api reference detailing all functionality.

Device Discovery

For Sync Box API to be available, the device must first be connected on WIFI. First time setup must go through Hue Sync app. A third party app should ask the user if the Sync Box LED is white or red, indicating the WIFI is already connected. If it is blinking blue then the user should first use the Hue Sync app for initial setup.

When the device is connected to WiFi, it can be found through mDNS. The service to look for is _huesync_tcp. The hostname equals the device unique id and the port is normally 443. The local ip address is currently

IPv4. The TXT record contains further information on the user given device name, devicetype (HSB1), uniqueid, and api path (/api). This information can be found [here](#).

Using the DNS-SD protocol to browse for services (Windows 10 & Mac)

Browse for all instances of the _huesync service:

```
dns-sd -B _huesync._tcp local.
```

Look up the information about the Sync box named HueSyncBox-C42996000000:

```
dns-sd -L "HueSyncBox-C42996000000" _huesync._tcp local.
```

Get the IP address for the given host name C42996000000:

```
dns-sd -G v4 C42996000000.local
```

Registration

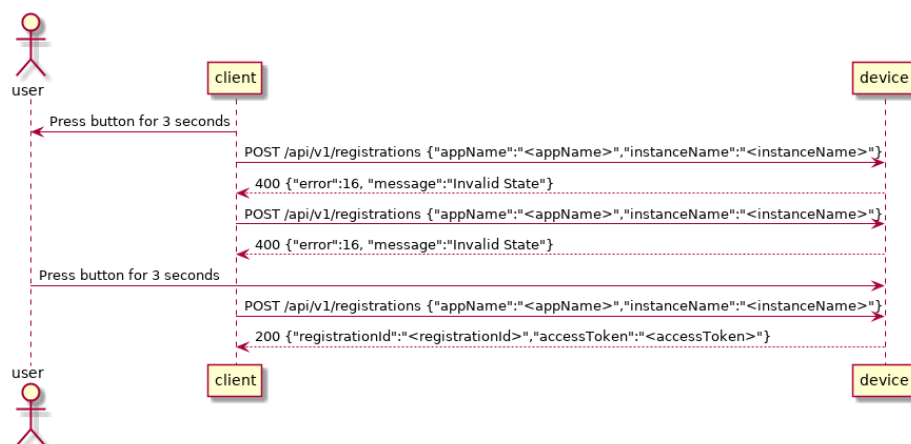
For most of API endpoints a registration is required. The following steps are needed for a client to create such registration.

Client tells users to press the button for 3 seconds until the LED blinks green to authorize this client to control Sync Box device.

Client tries to create a new registration repeatedly no less than every 5 seconds (typically every second)

When the user has authorized the client, the client receives a 200 response with an accessToken. This can be used as Bearer authorization token in regular api requests i.e. by including the following header in the request:

Authorization: Bearer <accessToken>



```
{  
  "registrationId": "1",  
  "accessToken": "<accessToken>"  
}
```

Authorized HTTP requests use Bearer authorization with the accessToken retrieved through pushlink,

The only two endpoints which can be called without authorization header are GET /api/v1/device to get basic device information and POST /api/v1/registrations to register a new app instance. All other endpoints require an authorization header.

HTTPS

The API only supports HTTPS requests. Always use HTTPS with certificate validation when sending requests that include the Authorization header.

The certificate validation must include checking the device server certificate against the Sync Box CA certificate, and validating that the common name matches the device unique id.

```
curl --cacert hsb_cacert.pem -H "Authorization: Bearer <token>"  
-X GET https://<id>/api/v1 --resolve "<id>:443:<ip>"
```

Click [here](#) to download `hsb_cacert.pem`. It should be pinned in the client application.

To verify the certificate using this CA, you typically add it to a custom keystore, and pass a trustmanager to your http client that trusts the certificates in that store.

To validate the common name, you would either have to make the request to the hostname and make sure your http client can resolve it to the ip (through mDNS or hardcoded), or if you make the requests to the ip directly, you would typically inject a custom hostname verifier which check that the server principal name matches the device unique id.

For performance reasons it is important to make sure consecutive requests share the same connection. This is best achieved by using a solid HTTP client library (for example curl or `okhttp`).

Example Configuration

```
{  
  "device": {  
    "name": "My Sync Box",  
    "deviceType": "HSB1",  
    "uniqueId": "C42996000000",  
  }  
}
```

```

    "apiLevel": 7,
    "firmwareVersion": "1.7.4",
    "buildNumber": 681947148,
    "wifiState": "wan",
    "ipAddress": "192.168.1.12",
    "wifi": {
      "ssid": "Wifi_2G",
      "strength": 4
    },
    "lastCheckedUpdate": "2020-02-16T11:17:13Z",
    "updatableBuildNumber": null,
    "updatableFirmwareVersion": null,
    "update": {
      "autoUpdateEnabled": true,
      "autoUpdateTime": 11
    },
    "ledMode": 1,
    "action": "none",
    "pushlink": "idle",
    "capabilities": {
      "maxIrCodes": 24,
      "maxPresets": 16
    }
  },
  "hue": {
    "bridgeUniqueId": "001788FFFE000000",
    "bridgeIpAddress": "192.168.1.8",
    "groups": {
      "db7dd240-d061-48bf-84c2-01f086e4bfae": {
        "name": "TV Area",
        "numLights": 5,
        "active": false
      },
      "f7bd7dcb-bbcb-4cd1-b343-126e60575884": {
        "name": "PC Area",
        "numLights": 4,
        "active": false
      }
    },
    "connectionState": "connected"
  },
  "execution": {
    "mode": "powersave",
    "syncActive": false,
    "hdmiActive": false,
    "hdmiSource": "input1",
    "hueTarget": "db7dd240-d061-48bf-84c2-01f086e4bfae",
    "brightness": 122,
    "lastSyncMode": "video",
    "video": {
      "intensity": "moderate",
      "backgroundLighting": true
    }
  },

```

```
"game": {
  "intensity": "high",
  "backgroundLighting": false
},
"music": {
  "intensity": "high",
  "palette": "melancholicEnergetic"
},
"preset": null
},
"hdmi": {
  "input1": {
    "name": "HDMI 1",
    "type": "generic",
    "status": "plugged",
    "lastSyncMode": "video"
  },
  "input2": {
    "name": "Gaming",
    "type": "xbox",
    "status": "plugged",
    "lastSyncMode": "game"
  },
  "input3": {
    "name": "HDMI 3",
    "type": "generic",
    "status": "unplugged",
    "lastSyncMode": "music"
  },
  "input4": {
    "name": "Shield",
    "type": "shield",
    "status": "plugged",
    "lastSyncMode": "video"
  },
  "output": {
    "name": "HDMI Out",
    "type": "generic",
    "status": "plugged",
    "lastSyncMode": "video"
  },
  "contentSpecs": "3840 x 2160 @ 60000 - SDR",
  "videoSyncSupported": true,
  "audioSyncSupported": true
},
"behavior": {
  "inactivePowersave": 20,
  "cecPowersave": 1,
  "usbPowersave": 1,
  "hpdInputSwitch": 1,
  "hpdOutputEnableMs": 1500,
  "arcBypassMode": 0,
  "forceDoviNative": 0,
  "input1": {
```

```

        "cecInputSwitch": 1,
        "linkAutoSync": 0,
        "hdrMode": 0
    },
    "input2": {
        "cecInputSwitch": 1,
        "linkAutoSync": 0,
        "hdrMode": 0
    },
    "input3": {
        "cecInputSwitch": 1,
        "linkAutoSync": 0,
        "hdrMode": 0
    },
    "input4": {
        "cecInputSwitch": 1,
        "linkAutoSync": 0,
        "hdrMode": 0
    }
},
"ir": {
    "defaultCodes": true,
    "scan": {
        "scanning": false,
        "code": null,
        "codes": []
    },
    "codes": {}
},
"registrations": {
    "1": {
        "appName": "Hue Sync iOS",
        "instanceName": "iPhone X",
        "role": "user",
        "lastUsed": "2020-02-08T02:21:49Z",
        "created": "2020-01-21T02:26:24Z"
    },
    "0": {
        "appName": "Hue Sync Android",
        "instanceName": "Pixel",
        "role": "admin",
        "lastUsed": "2020-02-16T05:45:20Z",
        "created": "2020-01-11T05:45:20Z"
    }
},
"presets": {}
}

```

General Concepts

The Sync Box API can be accessed locally via HTTPS on root level (port 443, /api/v1), resource level /api/v1/<resource> and in some cases

subresource level `/api/v1/<resource>/<subresource>`. This means that resources and subresources can either be in the path when they are accessed directly, or show up as a JSON key for an object as part of a higher level tree structure.

PUT requests are atomic at the resource level: multiple attributes can be modified at the same time, but the full request either fully fails or fully succeeds.

Almost all requests have a generic list of possible responses based on the request method. These are listed in the next section, so not repeated per resource in the table. However, if there is a specific response (e.g. POST `/registrations`), then it will be explained at the respective resource.

Some attributes have additional format requirements on top of the generic JSON type.

- free: means free format i.e. no additional format restrictions
- enum: can only be one of the values mentioned in the values column
- uint: must be a whole integer (no fraction) and ≥ 0
- hex: only contain characters `<0-9>` and `<A-F>`
- base64: only contain characters `<0-9>`, `<a-z>`, `<A-Z>`, `'>`, `'>` and `'>` for padding
- ip: must be of format `<0-255>.<0-255>.<0-255>.<0-255>`
- version: must start with `<number>.<number>`
- time: ISO 8601 formatted (YYYY-mm-ddTHH:MM:SSZ)

Resource Table (apiLevel 10)

Device

<code>/api/v1/device/..</code>	Methods	JSON Type, Format	Values
(device resource)	Get, Put	Object, Device	Root object for device resource
name	Get, Put	string, free	Friendly name of the device
deviceType	Get	string, enum	Device Type identifier – currently fixed to HSB1 Capitalized hex

uniqueId	Get	string, hex	string of the 6 byte / 12 characters device id without delimiters. Used as unique id on label, certificate common name, hostname etc.
ipAddress	Get	string, ip	Local IP address of the device
apiLevel	Get	number, uint	Increased between firmware versions when api changes. Only apiLevel >= 7 is supported.
firmwareVersion	Get	string, version	User readable version of the device firmware, starting with decimal major .minor .maintenance format e.g. "1.12.3"
updatableFirmwareVersion	Get	string, version	User readable version of the firmware the device can upgrade to. Item is set to null when there is no update available.
buildNumber	Get	number, uint	Build number of the firmware. Unique for every build with newer builds guaranteed a higher number than older.
updatableBuildNumber	Get	number, uint	Build number that is available to update to. Item is set to null when there is no update available.

lastCheckedUpdate	Get	string, time	UTC time when last check for update was performed.
wifiState	Get	string, enum	uninitialized, disconnected, lan, wan
wifi	Get	object	Root object for Wifi information
wifi/ssid	Get	string, free	Wifi SSID
wifi/strength	Get	number, enum	0 = not connected; 1 = weak; 2 = fair; 3 = good; 4 = excellent
ledMode	Get, Put	number, enum	1 = regular; 0 = off in powersave, passthrough or sync mode; 2 = dimmed in powersave or passthrough mode and off in sync mode
action	Get, Put	string, enum	none, doSoftwareRestart, doFirmwareUpdate
update	Get, Put	object	Root object for automatic update configuration
update/autoUpdateEnabled	Get, Put	boolean	Sync Box checks daily for a firmware update. If true, an available update will automatically be installed. This will be postponed if Sync Box is passing through content to the TV and being used.
			UTC hour when the

update/autoUpdateTime	Get, Put	number, uint	automatic update will check and execute, values 0 – 23. Default is 10. Ideally this value should be set to 3AM according to user's timezone.
overheating	Get	boolean	Indicates if the PSU voltage is too low. This is a critical error that should be displayed to user.
undervolt	Get	boolean	Indicates if the PSU voltage is too low. This is a critical error that should be displayed to user.
			Sync Box 8K only
bluetooth	Get, Put	boolean	Indicates current Bluetooth status whether it is enabled or disabled
capabilities	Get	object	Root object for capabilities resource
capabilities/maxIrCodes	Get	number, uint	The total number of IR codes configurable
capabilities/maxPresets	Get	number, uint	The total number of Presets configurable

hue

/api/v1/hue/..	Methods	JSON Type, Format	Values
		Object,	

(hue resource)	Get, Put	hue	Root object for hue resource
bridgeUniqueId	Get, Put	string, hex	16 character ascii hex string bridge identifier
bridgeIpAddress	Get	string, ip	Readable, dot IPv4 address of the paired bridge EG "192.168.1.50"
username	Put	string, free	randomly-generated username for Hue bridge, also referred to as application_key or hue-application-key
clientKey	Put	string, hex	32 character ASCII hex representation of 16 byte client key needed for streaming to hue entertainment
connectionState	Get	string, enum	uninitialized, disconnected, connecting, unauthorized, connected, invalidgroup, streaming, busy
groups	Get	object, map	All available entertainment areas on the current bridge. When this object is not available, it means the bridge areas have not been retrieved yet. When the object is empty, it means there are no entertainment areas on the bridge. When the bridge connection is lost, the last known values are remembered. Determining whether values may be outdated can be done based on connectionState.
groups/<id>	Get	object, group	
groups/<id>/name	Get	string, free	Friendly name of the entertainment area
groups/<id>/numLights	Get	number, uint	Number of lights in the entertainment area

groups/<id>/active	Get, Put	boolean	
groups/<id>/owner	Get	string, free	Only exposed if active is true

execution

		JSON	
/api/v1/execution/..	Methods	Type, Format	Values
(execution resource)	Get, Put	Object, execution	Root object for execution resource
syncActive	Get, Put	boolean	Reports false in case of powersave or passthrough mode, and true in case of video, game, music, or ambient mode. When changed from false to true, it will start syncing in last used mode for current source. Requires hue /connectionState to be connected. When changed from true to false, will set passthrough mode.
hdmiActive	Get, Put	boolean	Reports false in case of powersave mode, and true in case of passthrough, video, game, music or ambient mode. When changed from false to true, it will set passthrough mode. When changed from true to false, will set powersave mode.
mode	Get, Put	string, enum	powersave, passthrough, video, game, music, ambient (More modes can be added in the future, so

				clients must gracefully handle modes they don't recognize)
lastSyncMode	Get	string, enum		video, game, music, ambient
hdmiSource	Get, Put	string, enum		input1, input2, input3, input4 (currently selected hdmi input)
hueTarget	Get, Put	string, enum		Currently selected entertainment area (/groups/<id> for entertainment group on bridge api v1, and entertainment configuration <id> in UUID format for bridge api v2)
brightness	Get, Put	number, uint		0 – 200 (100 = no brightness reduction/boost compared to input, 0 = max reduction, 200 = max boost)
video	Get, Put	object, video		Root for video subresource
video/intensity	Get, Put	string, enum		subtle, moderate, high, intense
video/ backgroundLighting	Get, Put	boolean		
game	Get, Put	object, video		Root for game subresource
game/intensity	Get, Put	string, enum		subtle, moderate, high, intense
game/ backgroundLighting	Get, Put	boolean		
music	Get, Put	object, music		Root for music subresource
music/intensity	Get, Put	string, enum		subtle, moderate, high, intense

music/palette	Get, Put	string, enum	happyEnergetic, happyCalm, melancholicCalm, melancholic Energetic, neutral
toggleSyncActive	Put	boolean, true	true toggles syncActive
toggleHdmiActive	Put	boolean, true	true toggles hdmiActive
cycleSyncMode	Put	string, enum	next, previous
cycleHdmiSource	Put	string, enum	next, previous
incrementBrightness	Put	number, int	-200 – 200
cycleIntensity	Put	string, enum	next, previous (cycle intensity of current mode if syncing)
intensity	Put	string, enum	subtle, moderate, high, intense (if syncing)
preset	Get, Put	string	Preset identifier, that will be executed

hdmi

/api/v1/hdmi/..	Methods	JSON Type, Format	Values
(hdmi resource)	Get, Put	Object, hdmi	Root object for hdmi resource
contentSpecs	Get	string, x@-	<horizontal pixels> x <vertical pixels> @ <framerate fpks> – <HDR>
videoSyncSupported	Get	boolean	Current content specs supported for video sync (video/game mode)

audioSyncSupported	Get	boolean	Current content specs supported for audio sync (music mode)
<output/input1/2/3/4>	Get, Put	object, port	Root object for each of the 4 hdmi input subresources and output
<output/input1/2/3/4>/name	Get, Put	string, free	Friendly name, not empty
<output/input1/2/3/4>/type	Get, Put	string, enum	Friendly type: generic, video, game, music, xbox, playstation, nintendoswitch, phone, desktop, laptop, appletv, roku, shield, chromecast, firetv, diskplayer, settopbox, satellite, avreceiver, soundbar, hdmiswitch
<output/input1/2/3/4>/status	Get	string, enum	unplugged, plugged, linked, unknown
<output/input1/2/3/4>/lastSyncMode	Get	string, enum	video, game, music

Behavior

/api/v1/behavior/..	Methods	JSON Type, Format	Values
(behavior resource)	Get, Put	object, behavior	Root object for behavior resource
inactivePowersave	Get, Put	number, uint	Device automatically goes to powersave after this many minutes of being in passthrough mode with no link on any source or no link on output. 0 is disabled, max is 10000. Default: 20.

cecPowersave	Get, Put	number, enum	Device goes to powersave when TV sends CEC OFF. Default: 1. Disabled 0, Enabled 1.
usbPowersave	Get, Put	number, enum	Device goes to powersave when USB power transitions from 5V to 0V. Default: 1. Disabled 0, Enabled 1.
hpdInputSwitch	Get, Put	number, enum	Automatically switch input when any source is plugged in (or powered on). Default: 1. Disabled 0, Enabled 1.
forceDoviNative	Get, Put	number, enum	When the TV advertises Dolby Vision force to use native native mode. Disabled 0, Enabled 1.
input<1/2/3/4>	Get, Put	object, inputBehavior	Sync Box 4K only Root object for each of the 4 hdmi input subresources
input<1/2/3/4>/cecInputSwitch	Get, Put	number, enum	Automatically switch input when this source sends CEC active. Default: 1. Disabled 0, Enabled 1.
input<1/2/3/4>/linkAutoSync	Get, Put	number, enum	Automatically set syncActive true when this source and output are linked. Default: 0. Disabled 0, Enabled 1.

input<1/2/3/4>/ hdrMode	Get, Put	number, enum	HDR PQ compensation during Light Sync. 0 = Auto; 1 = Force SDR; 2 = Force HDR; Default: 0. Sync Box 4K only
input<1/2/3/4>/ hpdInputPortSwitch	Get, Put	number, enum	Automatically switch input when individual source is plugged in (or powered on). Default: 1 for all 4 hdmi input, Disabled 0, Enabled 1.

IR

/api/v1/ir/..	Methods	JSON Type, Format	Values
(ir resource)	Get, Put	object, ir	Root object for IR resource
scan			Scanning mode causes the last-received IR code to be saved (instead of processing), displayed as the 'code' attribute. Scanning automatically deactivates after 20 seconds but can continually be enabled again without gaps. After scanning an IR code, scanning will immediately be disabled.
scan/scanning	Get, Put	boolean	
scan/code	Get	string, hex	The last scanned code received while in scanning mode. Value is null if not scanned.
codes	Get, Put, Post	object	

codes/<code>	Get, Put, Delete	object	
codes/<code>/execution	Get, Put	execution	Execution object with only single item (i.e. only one key-value, if more items are needed then create a preset and use preset key to recall it by id)
codes/<code>/name	Get, Put	string	Friendly name intended for the user. Max length is 24 bytes

Registration

/api/v1/registrations/..	Methods	JSON Type, Format	Values
(registrations resource)	Get, Post	object, map	POST requires pushlink and provides registrationId and accessToken in response body, and might recycle the least recently used registration if required for storage.
<registrationid>	Get, Delete	object, registration	DELETE only the instance itself
<registrationid>/appName	Get, Post	string, free	User recognizable name of registered application
<registrationid>/instanceName	Get, Post	string, free	User recognizable name of application instance. Either a user name if single registration for user is shared over devices, or device name if each

			device uses a separate registration.
<registrationid>/created	Get	string, time	UTC time when this registration was created
<registrationid>/lastUsed	Get	string, time	UTC time when this registration was last used
<registrationid>/role	Get	string, enum	admin/user

Presets

/api/v1/presets/..	Methods	JSON Type, Format	Values
(presets resource)	Get, Put, Post	object, map	
<presetid>	Get, Post, Delete	object, preset	Each preset identifier is a random 8 character ASCII string. Max number of presets is defined in Capabilities, subject to change. Deleting also causes internal references (such as IR) to be cleaned up.
<presetid>/ name	Get, Put	string	Friendly name intended for the user. Max length is 24 bytes
<presetid>/lastUsed	Get	string, time	UTC time when this preset was last used
<presetid>/ execution	Get, Put	object, execution	Object to write to execution when preset is activated. May not contain the “preset” key itself (to prevent loops). The execution attribute is not printed with root, must

be read via GET presets.

Responses

GET

If request succeeded, Sync Box will respond 200 and a JSON payload corresponding to the URI accessed. If request failed. Sync Box will return a specified response code and a 401 JSON payload with an error object for more details EG {"error":1, "message":"Authentication failed"}.

Action Point	Response Code	Response object	Description
Request Successfully	200	none	The request has been processed successfully. A JSON payload corresponding to the accessed URI (and credentials) is returned.
Invalid URI Path	404	none	Accessing URI path which is not supported
Authentication failed*	401	error	If credentials are missing or invalid, errors out. *If credentials are missing, continues on to GET only the Configuration state when unauthenticated, to allow for device identification.
Internal error	500	none	Internal errors like out of memory

PUT

PUT requests on a resource are treated atomically, meaning the request is either entirely accepted or rejected (EG set mode Video and brightness 10000 will cause the entire request to be rejected). The response status and error codes below indicate if the request had succeeded or failed. HTTP PUT requests on the root to multiple resources are supported but not recommended: every resource is treated individually, and if not all status and error codes match a 207 multi-status is returned without further details. If request succeeded: Sync Box will respond 200, and a JSON payload of an empty object EG {}. If request failed: Sync Box will return a specified response code, and for

400/401 a JSON payload with an error object EG {"error":12, "message": "Invalid JSON"}.

Action Point	Response Code	Response object	Description
Request Successfully Processed	200	none	The request has been processed successfully
Invalid URI Path	404	none	Accessing URI path which is not supported
Authentication failed	401	error	Credentials are missing in the request, or they are not valid
Bad request	400	error	Client errors like JSON is not correct, key doesn't exists, value passed in is of wrong type or outside expected range, request cannot be fulfilled due to invalid state (EG start sync but hue is not configured yet)
Internal error	500	none	Internal errors on device side

POST

If request succeeded: Sync Box will respond 200, and a JSON payload, either empty or (if applicable) information on the corresponding newly generated asset.

If request failed: Sync Box will return a specified response code, and a JSON payload with an error code EG {"error":12, "message": "Invalid JSON"}.

Action Point	Response Code	Error Object	Description
Request Successfully Processed	200	created	The request has been processed successfully. A JSON object is returned as the payload, either empty or containing the newly generated asset.

Invalid URI Path	404	none	Accessing URI path which is not supported
Authentication failed	401	error	Credentials are missing in the request, or they are not valid
Bad request	400	error	Client errors like JSON is not correct, value passed in is of wrong type or outside expected range.
Internal error	500	none	Internal errors on device side

Error Object

error	message	Optional detailed explanation
1	Missing token	
2	Invalid token	
3	Unauthorized scope	
10	Missing body	
11	Too large body	
12	Invalid JSON	
13	Invalid key	The key does not exist on this path
14	Invalid type	The key exists but the value type does not match
15	Invalid value	The key exists and the value type matches, but it is not according to the specified format or range
16	Invalid state	The key and value are valid but the device is in a state where updating the key to this value is not possible

©2025 Signify Holding. All rights reserved.